

1. Core Idea

The **Weighted Average Method** is a **defuzzification technique** used to convert a fuzzy set into a specific crisp value.

Here, the fuzzy set (A) (the output of fuzzy inference) is the union of (M) fuzzy sets.

It is suitable for **fuzzy systems whose output membership functions are symmetric and regularly distributed**.

Main idea:

For each output fuzzy set (i.e., the values corresponding to the linguistic variables), take its maximum membership degree $\mu(x)$;

Use these maximum membership degrees as weights to perform a weighted average on the corresponding crisp values x ;

The calculation formula is as follows:

$$x^* = \frac{\sum [\mu(x_i) \times x_i]}{\sum [\mu(x_i)]}$$

Among them:

x_i

: The distinct values corresponding to each language variable (e.g., 60, 70, 80...)

$\mu(x_i)$

: The maximum membership degree corresponding to this value

The advantage of this method is its simplicity in computation, yielding results very close to the COA (center of gravity method) while requiring less computational effort.

2. Example Illustration

Let's illustrate the Weighted Average Method through two simple examples.

A.fuzzy set (A)

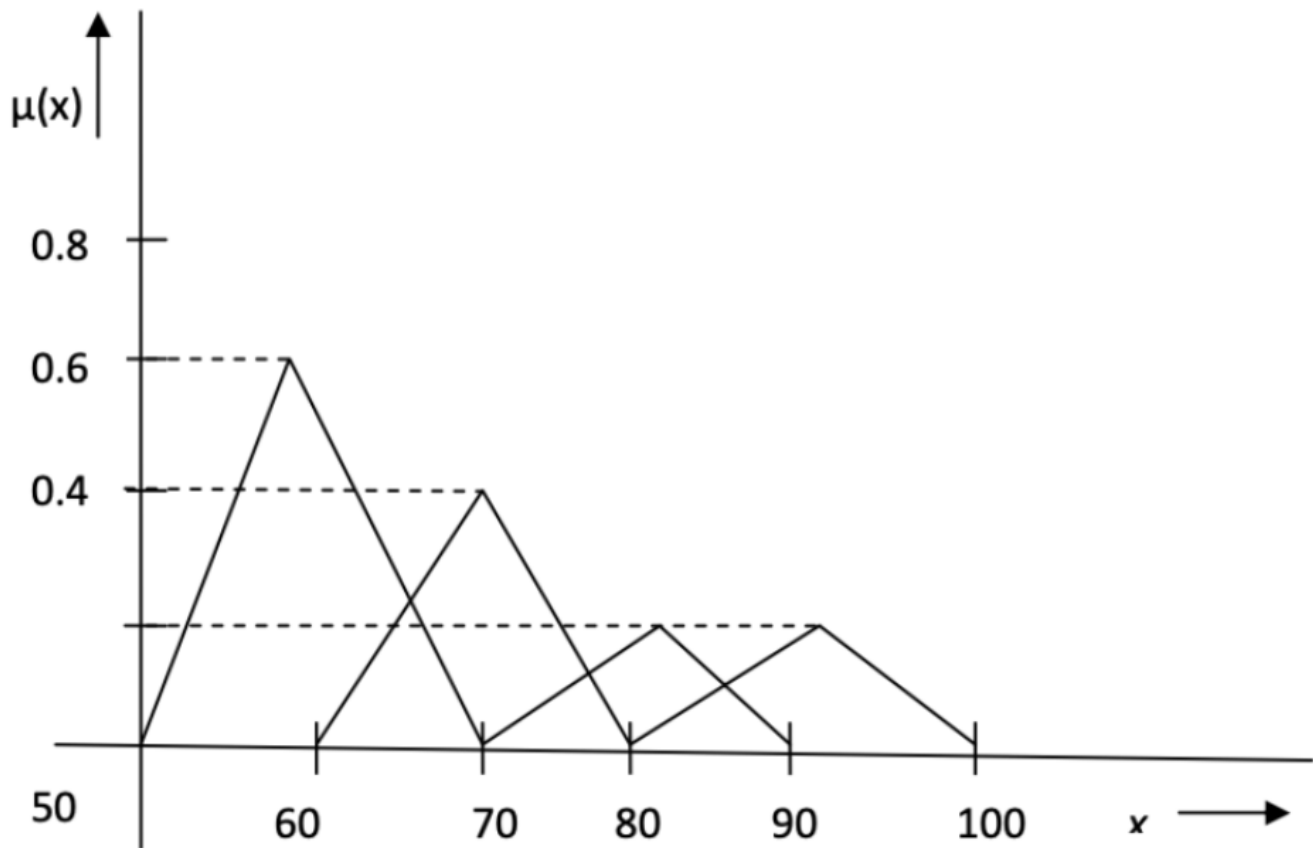
This is **not a complete fuzzy control system**, but rather an **example at the output layer**—we already have several fuzzy outputs (different grade levels and their membership degrees) and need to convert them into a **crisp score**.

Suppose fuzzy set (A) represents “*Student Performance Evaluation*”:

Linguistic Term	Meaning	x	$\mu(x)$
P	Pass	60	0.6
F	Fair	70	0.4
G	Good	80	0.2
VG	Very Good	90	0.2
E	Excellent	100	0.0

fuzzy set A

$$A = \{(P, 0.6), (F, 0.4), (G, 0.2), (VG, 0.2), (E, 0)\}$$



We design the output fuzzy set of “student grades” to be **regular, symmetric, and evenly distributed**, so that the defuzzification process is **more intuitive and easier to understand**, rather than fully representing a real-world case.

Applying the formula:

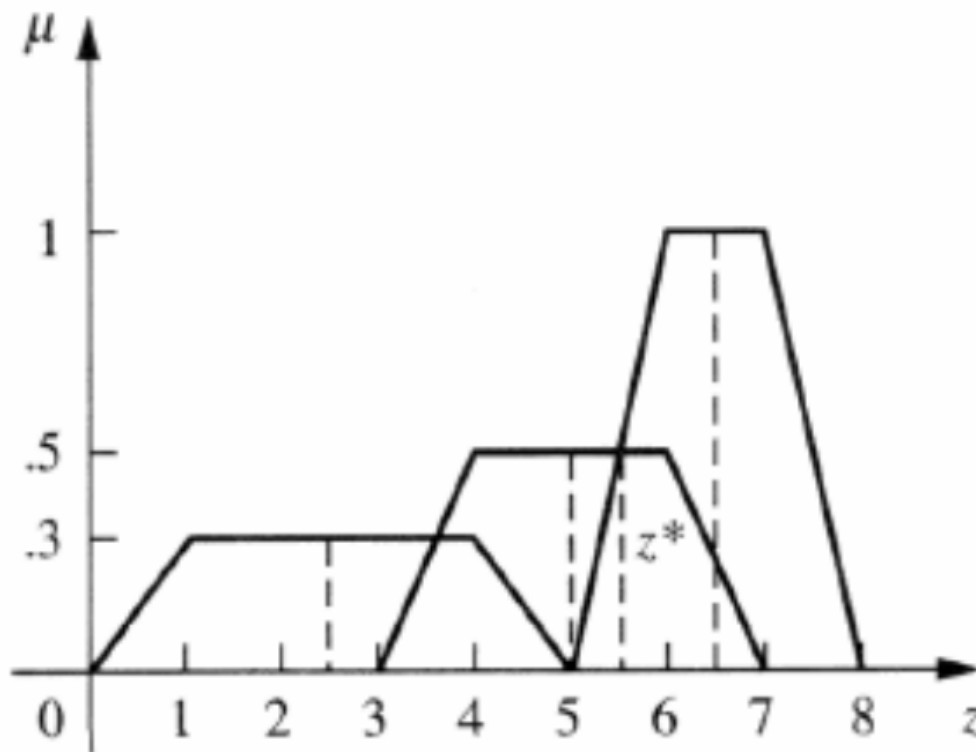
$$x^* = \frac{\sum [\mu(x_i) \times x_i]}{\sum [\mu(x_i)]}$$

$$x^* = \frac{60 \times 0.6 + 70 \times 0.4 + 80 \times 0.2 + 90 \times 0.2 + 100 \times 0}{0.6 + 0.4 + 0.2 + 0.2 + 0}$$

$$x^* = \frac{98}{1.4} = 70$$

So, the **defuzzified result** for the student's grade is **70 (Fair)**.

B.fuzzy set (Z)



$$z^* = \frac{(.3 \times 2.5) + (.5 \times 5) + (1 \times 6.5)}{.3 + .5 + 1} = 5.41 \text{ meters}$$

3. Python Implementation

Below is the Python code based on this A example, including both computation and visualization:

```
import numpy as np
import matplotlib.pyplot as plt

def triangular(x, a, b, c, height=1.0):
    """
    带高度参数的三角形隶属函数
    a: 左端点
    b: 顶点
    c: 右端点
    height: 顶点处的隶属度 (允许 < 1)
    """
    # 计算标准三角形 (顶点高度为1) 的值
    # 为避除以0, 处理退化区间
```

```

    left = np.where(b - a > 0, (x - a) / (b - a), 0.0)
    right = np.where(c - b > 0, (c - x) / (c - b), 0.0)
    mu_unit = np.maximum(np.minimum(left, right), 0.0)
    return height * mu_unit

# ----- x 网格 -----
x = np.linspace(50, 100, 1000)

# ----- 定义每个语言变量的三角形 -----
# 顶点位置: 60, 70, 80, 90, 100
# 对应的最大隶属度: 0.6, 0.4, 0.2, 0.2, 0.0
peaks = [60, 70, 80, 90, 100]
heights = [0.6, 0.4, 0.2, 0.2, 0.0]

# 为每个三角形选择左右端点 (每个三角宽度为20)
mus = []
for i, p in enumerate(peaks):
    a = p - 10    # 左端
    b = p         # 顶点
    c = p + 10    # 右端
    # 对于最右侧 E (peak at 100) 让右端不超过100
    if c > 100:
        c = 100
    mu = triangular(x, a, b, c, height=heights[i])
    mus.append(mu)

mu_P, mu_F, mu_G, mu_VG, mu_E = mus

# ----- 从图形中提取每个模糊子集的最大隶属度及其对应 x (自动) -----
mu_values = [np.max(mu) for mu in mus]
x_values = [x[np.argmax(mu)] if np.max(mu) > 0 else peaks[i] for i, mu in
             enumerate(mus)]
# 上面对 mu==0 的情况 (如Excellent) 用峰值位置作为 x 值 (但权重为0, 不影响计算)

# ----- 计算加权平均去模糊化 -----
mu_sum = np.sum(mu_values)
if mu_sum == 0:
    x_star = np.nan
else:
    x_star = np.sum(np.array(x_values) * np.array(mu_values)) / mu_sum

# ----- 输出数值 -----
print("Peaks (x):", peaks)
print("Heights (μ peaks):", heights)
print("Extracted x_values:", [round(v, 2) for v in x_values])
print("Extracted mu_values:", [round(v, 3) for v in mu_values])

```

```

print(f"Defuzzified value (x*): {x_star:.2f}")

# ----- 绘图 -----
plt.figure(figsize=(9,5))
plt.plot(x, mu_P, label='P (Pass)', linewidth=2)
plt.plot(x, mu_F, label='F (Fair)', linewidth=2)
plt.plot(x, mu_G, label='G (Good)', linewidth=2)
plt.plot(x, mu_VG, label='VG (Very Good)', linewidth=2)
plt.plot(x, mu_E, label='E (Excellent)', linewidth=2)

# 标出每个子集的顶点和最大隶属度的虚线
for xi, mui, lab in zip(x_values, mu_values, ['P', 'F', 'G', 'VG', 'E']):
    if mui > 0:
        plt.plot(xi, mui, 'ko') # 标点
        plt.hlines(mui, 50, xi, colors='gray', linestyle='dashed',
linewidth=1)
        plt.vlines(xi, 0, mui, colors='gray', linestyle='dotted',
linewidth=1)

# 去模糊化结果线
if not np.isnan(x_star):
    plt.axvline(x_star, color='red', linestyle='--', linewidth=1.5,
label=f'Defuzzified x* = {x_star:.2f}')

plt.xlim(50, 100)
plt.ylim(-0.02, 0.72)
plt.xlabel('x (Score)')
plt.ylabel('Membership degree  $\mu(x)$ ')
plt.title('Fuzzy sets (triangles) and Weighted Average defuzzification')
plt.grid(True, linestyle=':', linewidth=0.5)
plt.legend()
plt.show()

```

4. Results and Visualization

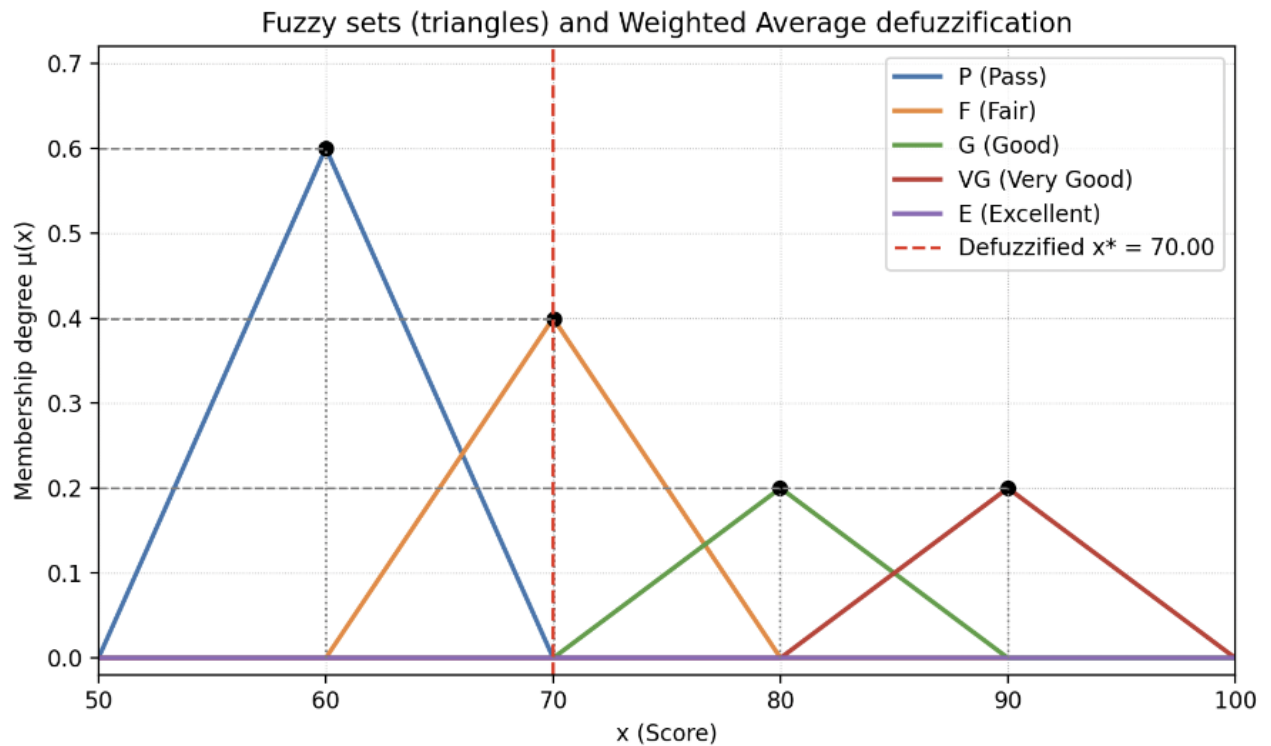
Output:

Defuzzified value (x*): 70.0

The figure shows:

- Five triangular membership functions
- Dashed gray lines marking each subset's peak and max membership

- A red dashed line indicating the **defuzzified result** ($x^* = 70$)



5. Summary

Item	Content
Method Name	Weighted Average Method
Applicable Scenario	Symmetric and regular output membership functions
Advantage	Simple calculation, results close to COA
Disadvantage	Less accurate for asymmetric functions